Co2_Assessment Tool-2_Set 3

Digital Logic — Debugging Analysis

Name:Hasmath Nisha M

Register no:192524513

1. Signed Binary Subtraction (2's Complement) — Incorrect Results

Debugging the Process

Common failure points:

- **Complement generation error:** Taking the 1's complement and forgetting to add 1 (or adding 1 twice), giving off-by-one results.
- **Overflow misdetection:** Using carry-out alone to flag overflow instead of comparing carry-into-MSB vs. carry-out-of-MSB. Correct rule: overflow = carry_in(MSB) XOR carry_out(MSB).
- **Sign-bit mishandling:** Dropping the final carry-out of the MSB, which is discarded in 2's complement addition (unlike 1's complement, where it is added back).

Recommended Corrections

- Convert subtraction A − B to A + (−B), where −B = invert(B) + 1.
- Detect overflow using the XOR rule above, never from carry-out alone.
- Ensure sufficient bit-width so the true magnitude fits within $[-2^{n-1}, 2^{n-1} - 1]$.

2. Carry Look-Ahead Adder — Incorrect Sum for Certain Inputs

Debugging the Generate/Propagate Logic

- **Generate vs. propagate confusion:** Gi = Ai·Bi and Pi = Ai⊕Bi (or Ai+Bi in some designs) — mixing OR/XOR forms breaks specific input patterns.
- **Incorrect carry recurrence:** Ci+1 = Gi + Pi·Ci — an error here (e.g., using AND instead of OR) causes wrong carries only for patterns exercising multi-level propagation.
- **Fan-in/timing limits:** Beyond 4 bits, look-ahead logic is normally hierarchical (block CLA); failing to properly combine block-generate/propagate terms causes errors at block boundaries.

Recommended Modifications

- Re-derive Gi and Pi per bit and verify the recurrence relation symbolically on a small (4-bit) case.
- For wider adders, use a hierarchical (2-level) CLA with block G/P terms rather than one flat look-ahead network — this restores correctness at block boundaries while preserving high-speed operation.

3. Booth's Algorithm Multiplier — Excess Clock Cycles

Debugging the Execution Sequence

Booth's algorithm reduces cycles by examining bit pairs (Qn, Qn−1) and skipping the add/subtract step when consecutive bits are identical (00 or 11 — shift only). Excess cycles typically indicate:

- The implementation performs ordinary shift-add multiplication with Booth recoding added on, rather than genuinely skipping runs of identical bits.
- Radix-2 Booth is used, which still requires one cycle per bit even when correct.
- Improper register updates (Qn−1 not shifted correctly) forcing redundant add/subtract operations.

Optimization Without Sacrificing Accuracy

- Verify the bit-pair decode table: 00 → shift only; 01 → add, then shift; 10 → subtract, then shift; 11 → shift only.
- Upgrade to Radix-4 (modified) Booth encoding, which examines 3 bits at a time and halves the number of iterations, improving throughput without affecting final accuracy.

4. Non-Restoring Division — Incorrect Quotient Values

Debugging the Division Process

- **Sign-based branching error:** If the partial remainder is positive, the next step subtracts the divisor; if negative, it adds the divisor. A sign-check bug here flips this logic and cascades errors through all subsequent steps.
- **Quotient bit assignment:** Each step should set the quotient bit to 1 if the remainder is ≥ 0, otherwise 0. A common bug inverts this condition.
- **Missing final correction:** If the last partial remainder is negative, one final restoring addition of the divisor is required to obtain the true remainder — omitting this leaves the remainder incorrect even when the quotient is right.

Suggested Corrections

- Trace the algorithm step-by-step against a known example, confirming the add/subtract decision is based on the sign of the previous remainder.
- Confirm quotient bits are assigned correctly at each step.
- Apply the final restoring correction add whenever the last remainder is negative.

5. IEEE 754 Double-Precision — Performance Degradation in Graphics Processing

Debugging the Arithmetic Overhead

- Double precision (64-bit) requires more execution cycles per operation and more memory bandwidth than single precision (32-bit) — accuracy beyond what graphics rendering actually needs (rendering rarely requires more than ~7 decimal digits).
- Excess normalization/rounding cycles occur when operations are not batched, and unnecessary conversions between formats add overhead.

Recommended Optimization Techniques

- Switch to single-precision (float32), or half-precision (float16) where tolerance allows — standard practice in shaders and graphics pipelines.
- Use fused multiply-add (FMA) units to reduce the total operation count.
- Leverage SIMD/vector units to batch floating-point operations.
- Avoid unnecessary double ↔ float conversions within the pipeline.